МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ И МАССОВЫХ КОММУНИКАЦИЙ

Ордена Трудового Красного Знамени
Федеральное государственное бюджетное общеобразовательное учреждение высшего образования

“Московский технический университет связи и информатики”
(МТУСИ)
Кафедра “Программная инженерия”

Лабораторная работа №3
по дисциплине “Введение в информационные технологии”

Выполнил: Студент группы
БПИ2503
Яричевский Даниил

Москва
2025

Цель работы

Получить практические навыки в использовании Postman для тестирования и разработки API, изучить основные методы HTTP, научиться создавать и отправлять запросы, а также анализировать ответы сервера. По завершении каждого задания студенты должны сделать скриншоты своих запросов и ответов для включения в отчет. Отчет должен также содержать краткий анализ полученных данных и выводы о выполненной работе.

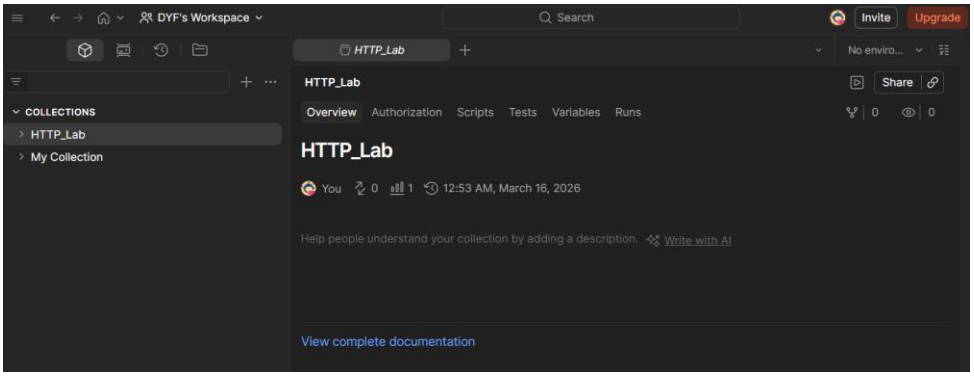
Задачи

1. Ознакомление с Postman и GET-запросами
2. Работа с query-параметрами
3. Отправка данных с использованием POST
4. Модификация данных с использованием PUT
5. Удаление данных с использованием DELETE
6. Удаление данных с использованием DELETE

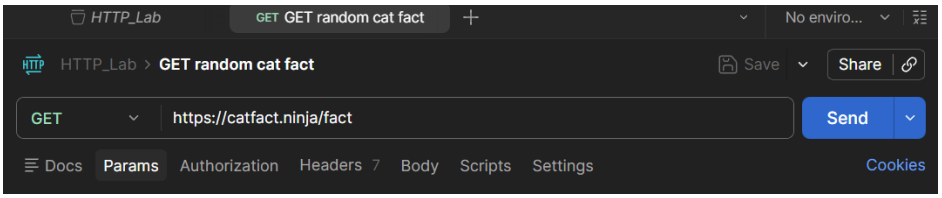
Ход работы

1.1 Установим Postman

1.2 Создим новую коллекцию с названием HTTP_Lab



1.3 Создадим GET-запрос к публичному API https://catfact.ninja/fact



1.4 Отправим запрос и изучим:

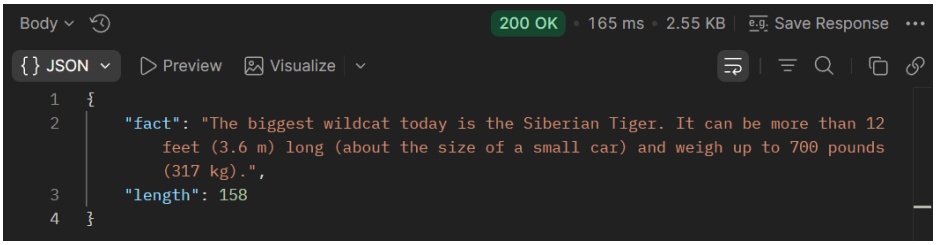
Статус-код ответа: 200 – Успешное выполнение запроса

Формат данных: JSON

Структура JSON-ответа:

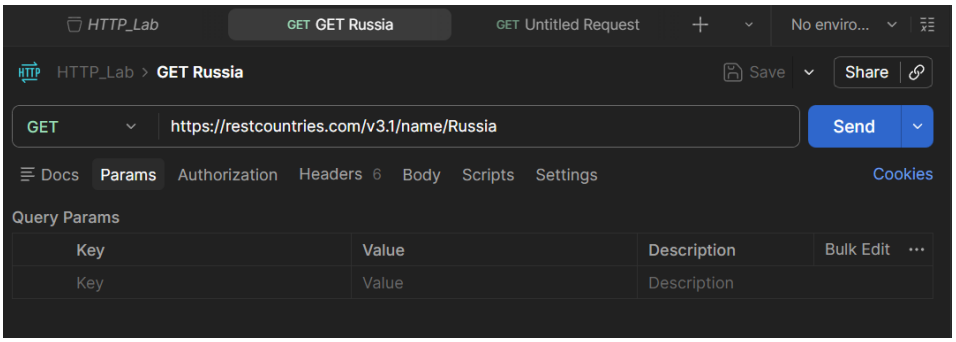
"fact" -текс со случайным фактом,

"length" – длина строки факта

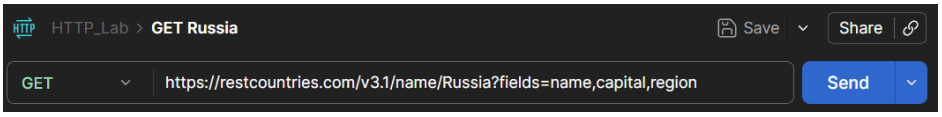


Вывод: Успешно вывели информацию с публичного ресурса

2.1 Выполним GET-запрос к API стран



2.2 Добавим query-параметр fields=name,capital,region



2.3 Сравним ответы сервера при наличии и отсутствии параметров

Без наличия: Полный вывод всего содержимого

При наличии: Только вывод полей name, capital, region

```
[
    {
        "tld": [
            ".ru",
            ".su",
            ".рф"
        ],
        "cca2": "RU",
        "ccn3": "643",
        "cca3": "RUS",
        "cioc": "RUS",
        "independent": true,
        "status": "officially-assigned",
        "unMember": true,
        "idd": {
            "root": "+7",
            "suffixes": [
                "3",
                "4",
                "5",
                "8",
                "9"
            ]
        },
        "capital": [
            "Moscow"
        ],
        "altSpellings": [
            "RU",
```

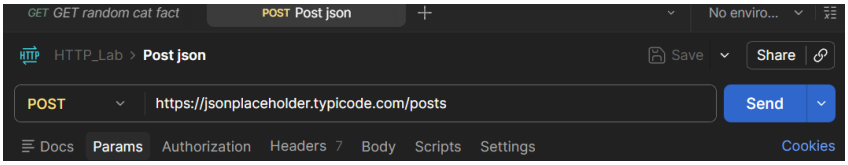

Body 200 OK 134 ms

JSON Preview Visualize

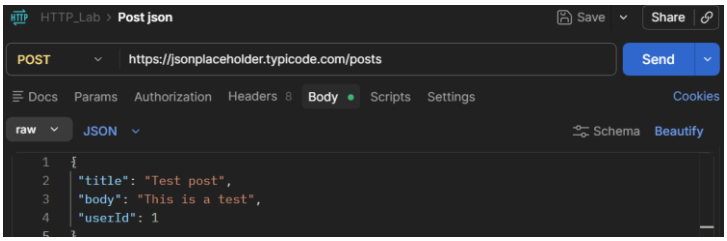

```
[
    {
        "name": {
            "common": "Russia",
            "official": "Russian Federation",
            "nativeName": {
                "rus": {
                    "official": "Российская Федерация",
                    "common": "Россия"
                }
            }
        },
        "capital": [
            "Moscow"
        ],
        "region": "Europe"
    }
]
```


Вывод: Сделали выборку того, что будем выводить с ресурса

3.1 Создадим POST-запрос к API



3.2 В разделе Body → raw → JSON передадим тело запроса:



3.3 Отправить запрос и проанализировать ответ сервера

Статус код-ответа: 201 – Успешно создано

Формат данных: JSON

Структура JSON-ответа:

“title” – название поста

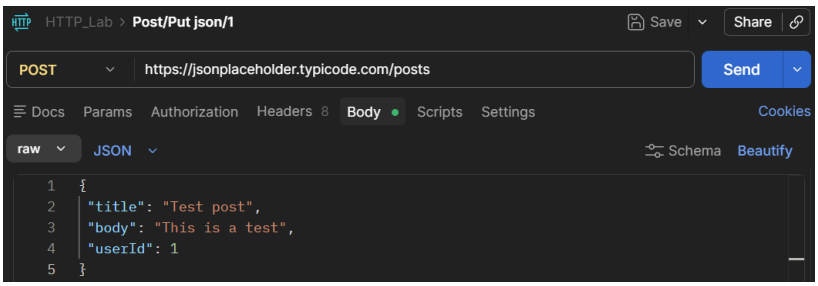
“length” – содержимое поста

“userId” – Id пользователя, кому принадлежит пост

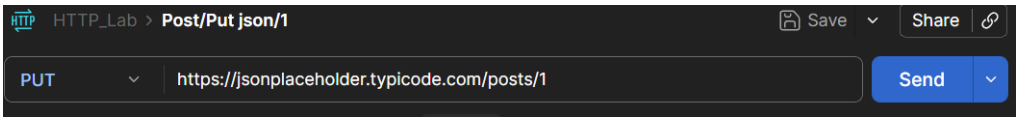
“id” – Id созданного поста

Вывод: Успешно создали новый пост с id 101

4.1 Создадим POST-запрос к API



4.2 Изменим метод на PUT и укажем полный URL для обновления поста



4.3 В "Body" изменим содержание поста, отправим запрос и проанализируем ответ:

Статус код-ответа: 200 – Успешное выполнение запроса

Формат данных: JSON

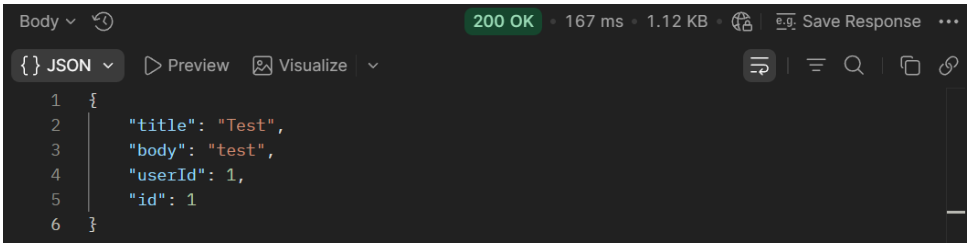
Структура JSON-ответа:

“title” – новое название поста

“length” – новое содержимое поста

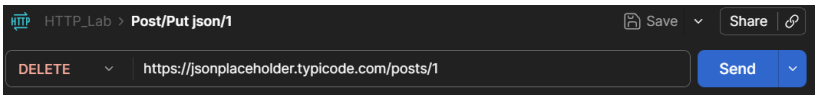
“userId” – Id пользователя, кому принадлежит пост

“id” – Id обновленного поста



Вывод: Успешно обновили пост с id 1

5.1 Выполним DELETE-запрос



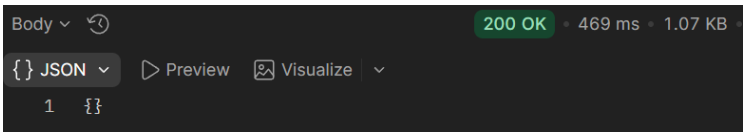
5.2 Зафиксируем статус-код ответа

Статус код-ответа: 200 – Успешное выполнение запроса

Формат данных: JSON

Структура JSON-ответа:

“{}” – Пустой



Вывод: Успешное удаление содержимого ресурса

5.3 Повторим GET-запрос к этому ресурсу

Статус код-ответа: 200 – Успешное выполнение запроса

Формат данных: JSON

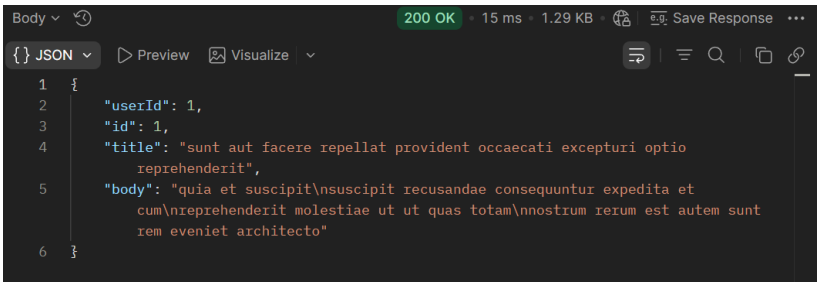
Структура JSON-ответа:

“title” – название поста

“length” – содержимое поста

“userId” – Id пользователя, кому принадлежит пост

“id” – Id выведенного поста



Вывод: Возврат исходных данных поста тк операция Delete не изменила данные на сервере

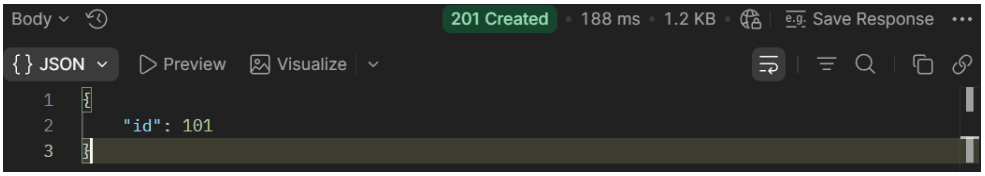
6.1 Выполним POST без тела запроса:

Статус код-ответа: 201 – Успешное создание

Формат данных: JSON

Структура JSON-ответа:

“id” – id созданного поста



Вывод: Вывод пустого поста с id 101

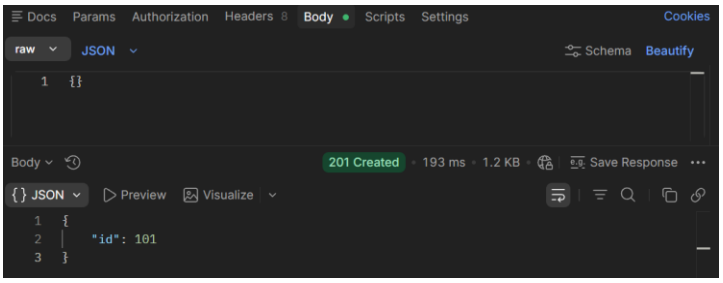
6.2 POST с пустым JSON {}

Статус код-ответа: 201 – Успешное создание

Формат данных: JSON

Структура JSON-ответа:

“id” – id созданного поста



Вывод: Вывод пустого поста с id 101

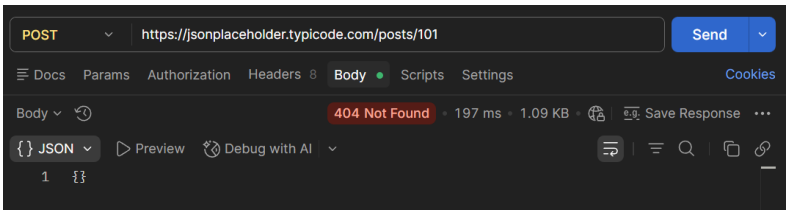
6.3 Выполним PUT к несуществующему ресурсу

Статус код-ответа: 404 – Запрашиваемый ресурс не найден (Ошибка Пользователя)

Формат данных: JSON

Структура JSON-ответа:

“{}” – Пустой



Вывод: Не удалось вывести содержимое ресурса тк оно не существует

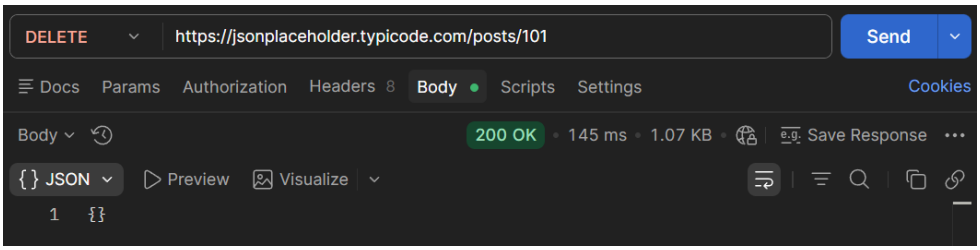
6.4 Выполним DELETE с неверным идентификатором

Статус код-ответа: 200 – Успешное выполнение

Формат данных: JSON

Структура JSON-ответа:

“{}” – Пустой



Вывод: DELETE выполнился тк ресурс отсутствовал и без этого

Вывод

Я получил практические навыки в использовании Postman для тестирования и разработки API, изучил основные методы HTTP, научился создавать и отправлять запросы, а также анализировать ответы сервера.